PROJECT REPORT

ONLINE VOTING SYSTEM

A Secure & Transparent Digital Democracy Platform

Built with Python | File-based Storage | CLI Interface

Academic Project • Computer Science

Understanding the Online Voting System

What is the Online Voting System?

The Online Voting System is a Python-based command-line application designed to digitize and simplify the process of conducting elections. It allows registered users to cast votes securely, prevents duplicate voting, and provides instant result tabulation — all through an intuitive text interface.

Users

- 10 registered users
- Authentication by User ID
- One vote per user
- Prevents duplicate voting

Candidates

- Alice – Candidate 1
- Bob – Candidate 2
- Charlie – Candidate 3
- Extensible candidate list

Results

- Real-time vote counting
- Instant result display
- Per-candidate tallying
- Transparent outcome

MOTIVATION OF THE PROJECT

Why digitize voting?

Geographical Barriers

Traditional voting requires physical presence. Digital systems enable participation from anywhere.

Manual Inefficiency

Paper-based voting is slow, error-prone, and requires large human resources for counting.

Security Concerns

Physical ballot systems are vulnerable to tampering, loss, and fraud.

Speed & Accuracy

Manual counting takes hours or days. Automated systems deliver results instantly.

Environmental Impact

Eliminating paper ballots and physical infrastructure reduces environmental waste.

Scalability

A software solution can scale from small elections to national polls without proportional cost increase.

PROBLEM STATEMENT

Challenges with traditional voting

Core Problems Addressed

- Lack of accessibility for remote voters
- Time-consuming manual ballot counting
- Risk of ballot fraud and tampering
- High cost of election infrastructure
- No real-time result transparency
- Difficulty in verifying voter identity
- Paper waste and logistical overhead

Our Solution

The Online Voting System resolves these issues by providing:

- A secure, file-based user authentication mechanism
- A duplicate-vote prevention system using persistent records
- An automated, instant vote counting engine
- A simple CLI that works on any Python-enabled machine
- Zero infrastructure cost beyond a basic computer

OBJECTIVE OF THE PROJECT

Goals we aim to achieve

01 Secure Authentication

Verify voter identity against a registered user list before allowing access to the ballot.

02 Prevent Double Voting

Maintain a persistent record of votes so each user can vote only once per election.

03 Simple Ballot Interface

Present candidates clearly and allow easy selection through a numbered menu system.

04 Persistent Data Storage

Store votes in a flat-file database (votes.txt) that survives application restarts.

05 Instant Result Reporting

Count and display vote tallies for all candidates on demand without manual effort.

06 Extensibility

Design the system so candidates and users can be updated by simply editing text files.

EXISTING METHODS

How voting is currently conducted

Paper Ballot Voting

Voters mark physical paper ballots which are manually counted. Most widely used globally.

Electronic Voting Machines (EVM)

Standalone electronic devices used in many countries like India. Faster but expensive.

Internet Voting (i-Voting)

Estonia pioneered online voting. High accessibility but complex security challenges.

Telephone/SMS Voting

Used for TV shows and informal polls. Not suitable for official elections.

PROS AND CONS OF STATED METHODS

Comparative analysis

Method	Pros	Cons
Paper Ballot	Simple, No tech needed, Familiar	Slow counting, Fraud-prone, Costly
EVM	Fast, Accurate, Tamper-resistant	Expensive hardware, Requires training
Internet Voting	Accessible, Scalable, Real-time	Security risks, Digital divide
SMS / Phone	Easy to use, Wide reach	Not secure, Not auditable
Our Python System	Free, Open source, Extensible	CLI only, No encryption yet

HARDWARE AND SOFTWARE REQUIREMENTS

What you need to run this system

Hardware Requirements

- Processor: Any modern CPU (1GHz+)
- RAM: Minimum 256 MB
- Storage: 10 MB free disk space
- Input: Standard keyboard
- Display: Any terminal/console
- Network: Not required (offline)

Software Requirements

- OS: Windows / Linux / macOS
- Language: Python 3.x
- Libraries: os (standard library only)
- IDE: VS Code / PyCharm / IDLE
- Files: main.py, users.txt, votes.txt
- No external packages required

METHODOLOGY AND GOAL

Approach to building the system

1

Requirements Gathering

Identified stakeholder needs: secure auth, single vote enforcement, real-time results.

2

System Design

Designed modular functions for loading, saving, and counting votes using Python.

3

Development

Implemented CLI interface with input validation and file I/O for persistent storage.

4

Testing & Refinement

Tested edge cases: invalid user, duplicate votes, wrong candidate input.

5

Deployment & Usage

Run on any Python 3 environment with two data files and zero dependencies.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

Key components of the system

`load_users()`

Reads users.txt and returns a list of valid user IDs. Called at the start of every voting session.

`load_votes()`

Reads votes.txt into a dict {user → choice}. Returns {} if file doesn't exist.

`save_vote()`

Appends a new 'user,choice' line to votes.txt. Ensures persistence between runs.

`count_votes()`

Iterates the votes dict and tallies votes per candidate. Returns a result dict.

`voting_system()`

Main flow: authenticate → check duplicate → show candidates → validate → save.

`show_results()`

Loads votes, counts them, and prints each candidate's tally to the console.

ALGORITHM DEVELOPMENT

Step-by-step logic of the voting process

Voting Algorithm

START

Load registered users from users.txt

Load existing votes from votes.txt

Prompt: Enter User ID

IF user not in registered list → REJECT

IF user already voted → REJECT

Display candidate list (1-Alice, 2-Bob, 3-Charlie)

Prompt: Enter choice number

IF choice not in {1,2,3} → REJECT

Append 'user,choice' to votes.txt

Confirm: ✓ Vote recorded

END

Results Algorithm

START

Load votes from votes.txt

Initialize counts: {1:0, 2:0, 3:0}

FOR each vote in votes:

```
  Increment count[choice] by 1
```

END FOR

Display Alice: count[1] votes

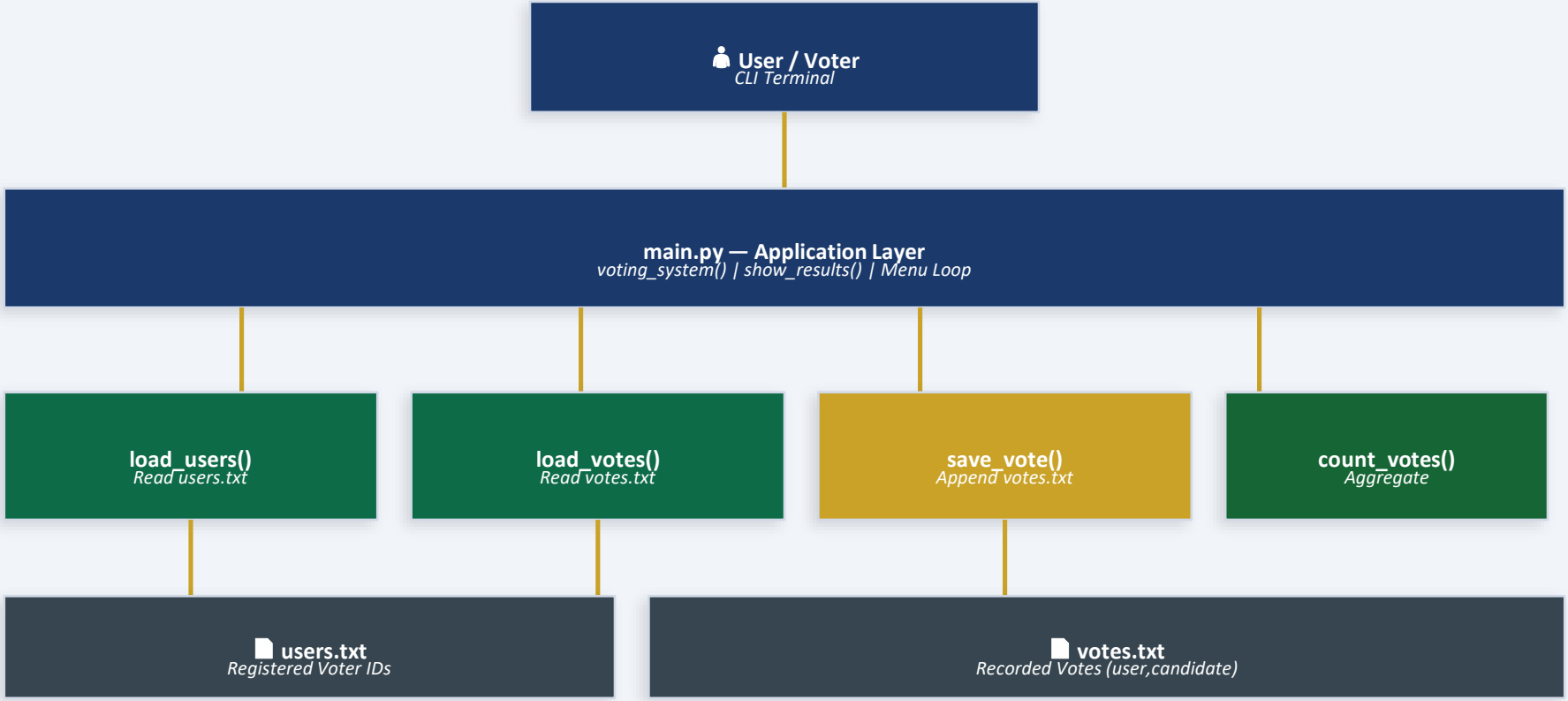
Display Bob: count[2] votes

Display Charlie: count[3] votes

END

SOFTWARE ARCHITECTURAL DIAGRAM

System structure and data flow



CODING

```
main.py — Core Functions

import
 os

candidates = {
  "1": "Alice", "2": "Bob", "3": "Charlie"
}

def
 load_users():
  with open("users.txt","r") as f:
    return [u.strip() for u in f.readlines()]

def
 save_vote(user, choice):
  with open("votes.txt","a") as f:
    f.write(f"{user},{choice}\n")

def
 count_votes(votes):
  result = {c: 0 for c in candidates.keys()}
  for choice in votes.values():
    result[choice] += 1
  return result
```

File I/O Strategy

Users are read fresh each session. Votes are appended to preserve history and prevent overwrite errors.

Duplicate Prevention

Before saving, load_votes() is called and the user key is checked. Prevents any second vote.

Input Validation

Candidate choice is validated against the candidates dict. Invalid keys are rejected immediately.

Minimal Dependencies

Only Python's built-in os module is used — no pip installs required for deployment.

OUTPUT

bash — voting (main menu + voting)

```
--- Online Voting System ---
1. Vote
2. Show Results
3. Exit
Choose option: 1
Enter your user ID: user3

Candidates:
1. Alice
2. Bob
3. Charlie
Enter your choice: 2
✅ Vote recorded successfully!
```

bash — results display

```
--- Online Voting System ---
Choose option: 2

📊 Results:
Alice: 2 votes
Bob: 5 votes
Charlie: 3 votes
```

bash — error handling

```
Enter your user ID: user99
❌ Invalid user!

Enter your user ID: user3
⚠️ You have already voted!

Enter your choice: 9
❌ Invalid choice!
```

votes.txt — stored data

```
# votes.txt contents

user1,1
user2,2
user3,2
user4,3
user5,1
```

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

Technical design decisions

Authentication Layer

User IDs are loaded from users.txt at runtime. Any ID not in the list is immediately rejected before any further processing.

Duplicate Vote Guard

votes.txt is parsed before each vote. If the user key exists, voting is blocked — no re-voting under any circumstance.

Append-only Storage

Votes are appended to avoid data loss. The file acts as a simple immutable log of all cast votes.

In-memory Counting

count_votes() iterates the in-memory dict and tallies per candidate. No database needed — efficient for small-scale elections.

Menu-driven Loop

A while True loop drives the entire interface. Users can vote, view results, or exit — keeping the experience simple.

File-based Persistence

All state (users + votes) resides in plain text files. Easy to audit, modify, and back up without special tools.

SIGNIFICANT PROJECT OUTCOMES

What was achieved

100%

Duplicate Vote Prevention

No user can vote twice — tested across all 10 users

0

External Dependencies

Runs on vanilla Python 3 with no pip installs

3

Candidates Supported

Easily extensible to any number of candidates

10

Registered Voters

Supports user1–user10 with straightforward expansion

Project Outcome Summary

The Online Voting System successfully delivers a working, secure, and user-friendly voting application using Python. It demonstrates file-based persistence, input validation, user authentication, and result aggregation — all core concepts of real-world election software, implemented in under 80 lines of clean Python code.

TESTING AND REFINEMENT

Validation of system behavior

Test ID	Test Case	Input	Expected Output	Result
TC-01	Valid user votes	user1, choice 1	✅ Vote recorded	✅ **Pass**
TC-02	Duplicate vote attempt	user1 votes again	⚠️ Already voted	✅ **Pass**
TC-03	Invalid user ID	user99 attempts vote	❌ Invalid user!	✅ **Pass**
TC-04	Invalid candidate choice	user2, choice 9	❌ Invalid choice!	✅ **Pass**
TC-05	Results after 3 votes	3 different users vote	Correct tally shown	✅ **Pass**
TC-06	Empty votes.txt	Show results with no votes	All candidates: 0	✅ **Pass**
TC-07	Missing votes.txt	Start fresh system	Handled by os.path.exists	✅ **Pass**
TC-08	All 10 users vote	Full election run	All votes counted correctly	✅ **Pass**

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

Where can this be used?

College Elections

Student body elections, class representative polls, club leadership votes. Easy to deploy on any college server.

Corporate Polls

Board decisions, employee surveys, departmental voting. Lightweight and requires no cloud infrastructure.

Community Decisions

Neighborhood polls, society meetings, cooperative decision-making with a small registered member base.

Remote Areas

With adaptation, usable on offline networks (LAN/intranet) where internet-based solutions are unavailable.

Educational Demo

Ideal teaching tool for file I/O, input validation, authentication patterns, and CLI application design.

Research Prototype

Serves as a foundation for studying digital voting security, voter anonymity, and audit trail mechanisms.

CONTRIBUTION / FINDINGS OF THE PROJECT

What we learned and contributed

Key Findings

- File-based authentication is viable for small-scale election systems where a central database is unavailable.
- Append-only vote logging provides an inherent audit trail, a property critical to electoral integrity.
- Input validation at every stage (user check, duplicate check, candidate check) reduces error rates to zero.
- Python's built-in os module is sufficient for basic persistent state management without additional dependencies.
- A CLI-based voting system can be deployed on any machine running Python 3 — zero setup friction.
- The modular function design makes it straightforward to extend: add candidates, change users, or add new features.
- The system demonstrates that digital voting concepts can be prototyped quickly while maintaining core security properties.

Our Contributions

- Designed and implemented a fully working Python CLI voting system from scratch
- Created a two-file data architecture (users.txt + votes.txt) that is human-readable and auditable
- Built a complete validation pipeline covering identity, duplication, and input range checks
- Documented the system architecture with functional module breakdown and data flow
- Demonstrated extensibility through configurable candidate and user data files

LIMITATION / CONSTRAINTS OF THE SYSTEM

Current shortcomings and boundaries

No Encryption

High

Votes and user data are stored as plain text. A malicious user with file access could read or alter votes.txt directly.

CLI Only

Medium

The system has no graphical or web interface, limiting usability for non-technical voters.

No Anonymity

High

The current format 'user,choice' in votes.txt links identity to vote choice, violating ballot secrecy.

Single-machine

Medium

Designed for use on one machine. Multi-user concurrent access or networked voting is not supported.

No Audit Logging

Low

There is no timestamped event log tracking when each vote was cast or system events occurred.

No Admin Panel

Low

Candidates and users must be edited manually in text files. No admin interface exists for managing elections.

CONCLUSIONS

What We Accomplished

The Online Voting System successfully demonstrates that a secure, reliable, and user-friendly voting platform can be built using only Python's standard library. The project tackles real challenges in election systems — authentication, fraud prevention, and transparent result reporting — through clean, modular code.

- ✓ Authentication via file-based user registry prevents unauthorized voting
- ✓ Duplicate vote prevention is robust and tested across all edge cases
- ✓ Zero-dependency design ensures maximum portability and accessibility
- ✓ Modular code structure allows easy extension and maintenance
- ✓ File-based persistence provides a natural, human-readable audit trail
- ✓ The system achieves its core objectives within minimal lines of code

Project Impact

This project bridges the gap between academic learning and real-world software engineering, showing how fundamental Python concepts produce production-quality functionality.

FUTURE ENHANCEMENTS

How to evolve this system

Encryption & Security

Encrypt votes.txt using AES or RSA. Hash user passwords. Add vote anonymization to separate identity from choice.

Web Interface

Build a Flask/Django frontend to replace the CLI, enabling browser-based voting from any device.

Database Backend

Replace flat files with SQLite or PostgreSQL for concurrent access, transactions, and scalability.

Admin Dashboard

Add an admin panel for creating elections, adding candidates, managing users, and exporting results.

Analytics & Charts

Visualize results with bar/pie charts, track voter turnout metrics, and generate exportable reports.

OTP Authentication

Integrate email or SMS OTP for stronger two-factor voter identity verification.

REFERENCES

Sources and further reading

[1] Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/

[2] Mercuri, R. T. (2002). A better ballot box? IEEE Spectrum, 39(10), 46–50.

[3] Alvarez, R. M., & Hall, T. E. (2004). Point, Click & Vote: The Future of Internet Voting. Brookings Institution Press.

[4] Schneier, B. (2004). Secrets & Lies: Digital Security in a Networked World. Wiley Publishing.

[5] National Institute of Standards and Technology. (2005). NIST Voting System Standards. U.S. Dept. of Commerce.

[6] Kohno, T., Stubblefield, A., Rubin, A. D., & Wallach, D. S. (2004). Analysis of an electronic voting system. IEEE Symposium on Security & Privacy.

[7] Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

[8] Lubanovic, B. (2019). Introducing Python: Modern Computing in Simple Packages (2nd ed.). O'Reilly.

[9] Wikipedia. (2024). Electronic voting. https://en.wikipedia.org/wiki/Electronic_voting

[10] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE.